hydra

/ˈhaɪdrə/

v0.5.0 2024-07-12 MIT

Query and display headings in your documents and templates.

tinger <me@tinger.dev>

https://github.com/tingerrr/hydra

Hydra provides a simple API to querying for headings und section like elements and displaying them in headers or footers. It exposes internal modules and functions for maximum control.

Table of contents

I. Manifest

II. Guide

II.1. Introduction 3
II.2. Terminology & Semantics 3
II.3. Features 4
II.4. Contextual 4
II.5. Custom Elements 4
II.6. Redundancy Checks 5
II.6.1. Starting Page 5
II.6.2. Book Mode 6
II.7. Anchoring 6

III. Reference

III.1. Stability 7
III.2. Custom Types 7
III.2.1. queryable 7
III.2.2. hydra-selector 7
III.2.3. full-selector 7
III.2.4. candidates 7
III.2.5. hydra-context 8
III.3. Modules 9
III.3.1. hydra 9
III.3.2. core 11
III.3.3. selectors 14
III.3.4. util 16
III.3.5. util/core 17
III.3.6. util/assert 18

IV. Index

Part I.
Manifest

Hydra aims to be:

- simple to use
  - importing a function and using it should be all that is needed
  - setup required to make the package work should be avoided
- unsurprising
  - parameters should have sensible names and behave as one would expect
  - deviations from this must be documented and easily accesible to Typst novices
- interoperable
  - Hydra should be easy to use with other packages by default or provide sufficient configuration to allow this in other ways
- minimal
  - it should only provide features which are specifically used for heading and section querying and display

If you think its behvior is surprising, you believe you found a bug or you think its defaults or parameters are not sufficient for your use case, please open an issue at GitHub:tingerrr/hydra. Contributions are also welcome!

Part II.
Guide

II.1. Introduction

Hydra is a package which aims to query and display section elements, such as headings,legal paragraphs, documentation sections, and whatever else may semantically declare the start of a document's section.

II.2. Terminology & Semantics

The following terms are frequently used in the remainider of this document.

primary The element which is primarily looked for and meant to be displayed.

ancestor An element which is the immediate or transitive ancestor to the primary element. A level 3 heading is ancestor to both level 2 (directly) and level 1 headings (transitively).

scope The scope of a primary element refers to the section of a document which is between the closest ancestors.

active The active element refers to whatever element is considered for display. While this is usually the previous primary element, it may sometimes be the next primary element.

leading page A leading page in a book is that, which is further along the content of the two visible pages at any time, this is the `end` alignment with respect to the document reading direction.

trailing page A trailing page is that, which is not the leading page in a book.

The search for a primary element is always bounded to its scope. For the following simplified document:

```
= Chapter 1
== Section 1.1

= Chapter 2
=== Subsection 2.0.1
#hydra(2)
```

```
Chapter 1
└ Section 1.1
Chapter 2
└ <none>
  └ Subsection 2.0.1
```

Hydra will only search within its current chapter as it is looking for active sections. In this case Hydra would not find a suitable candidate. For this the ancestors of an element must be known. For headings this is simple:

`<none>` ⟶ `level: 1` ⟶ `level: 2` ⟶ `level: 3` ⟶ `...`

If Hydra is used to query for level 2 headings it will only do so within the bounds of the closest level 1 headings. In principle, elements other than headings can be used (see Section II.5), as long as their semantic relationships are established.

II.3. Features

II.4. Contextual

Hydra will take contextual information into account to provide good defaults, such as inferring the reading direction and binding from the page and text styles to offer correct handling of books as seen in Section II.6.2.

II.5. Custom Elements

Because some documents may use custom elements of some kind to display chapters or section like elements, Hydra allows defining its own selectors for tight control over how elements are semantically related.

Given a custom element like so:

```
#let chapter = figure.with(kind: "chapter", supplement: [Chapter])
// ... show rules and additional setup

#chapter[Introduction]
#chapter[Main]
= Section 1.1
== Subsection 1.1.1
= Section 1.2
#chapter[Annex]
```

A user my want to query for the current chapter and section respectively:

```
#import "@preview/hydra:0.5.0": hydra, selectors
#import selectors: custom

#let chap = figure.where(kind: "chapter")
#let sect = custom(heading.where(level: 1), ancestor: chap)

#set page(header: context if calc.odd(here().page()) {
  align(left, hydra(chap))
} else {
  align(right, hydra(sect))
})
```

The usage of `custom` allows specifying an element's ancestors, to ensure the scope is correctly defined. The selectors module also contains some useful default selectors.

II.6. Redundancy Checks

Generally Hydra is used for heading like elements, i.e. elements which semantically describe a section of a document. Whenever Hydra is used in a place where its output would be redundant, it will not show any output by default. The following sections explain those checks more closely and will generally assume that Hydra is looking for headings for simplicity.

II.6.1. Starting Page

Given a page which starts with a primary element, it will not show anything. If `skip-starting` is set to `false`, it will fallback to the next element, in this case the heading at the top of the page.

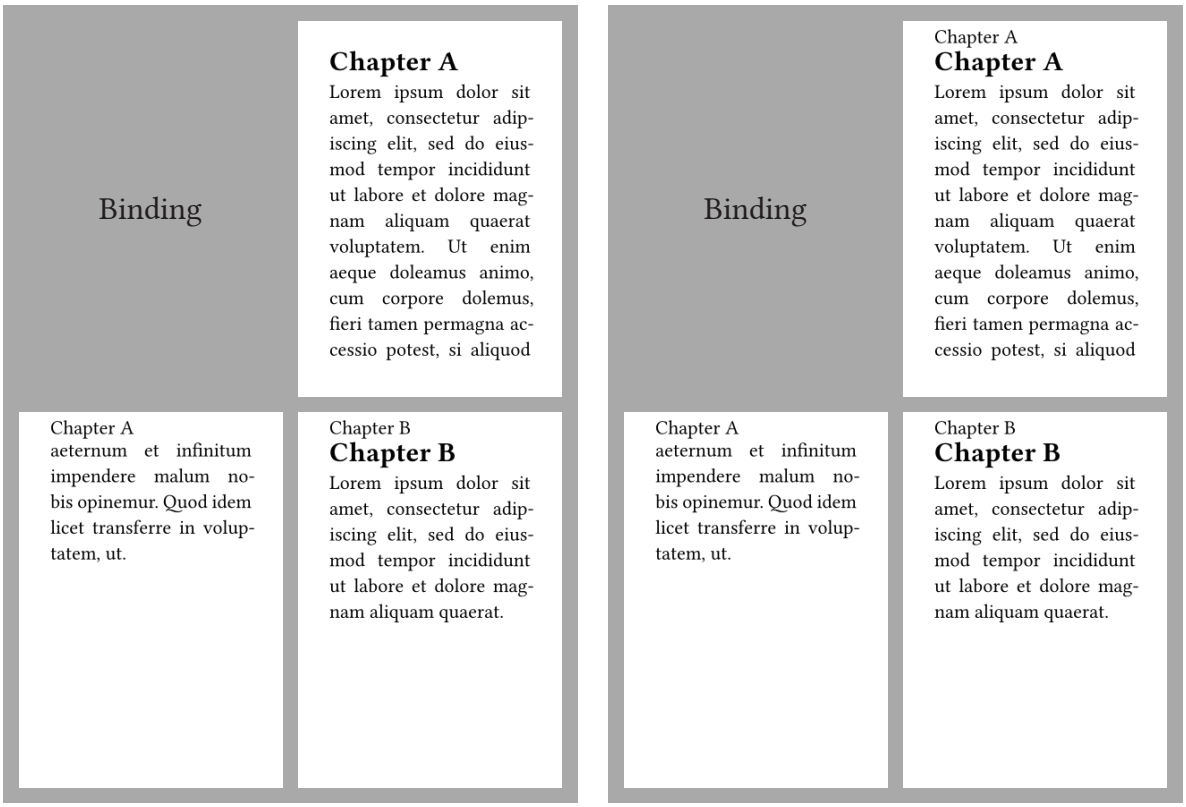

Figure 1: An example document showing `skip-starting: true` (left) and `skip-starting: false` (right).

For more complex selectors this will not correctly work if the first element on this page is an ancestor See hydra#8.

II.6.2. Book Mode

Given a leading page, if `book` is set to `true`, then if the previous primary element is still visible on the previous (trailing) page it is also skipped.

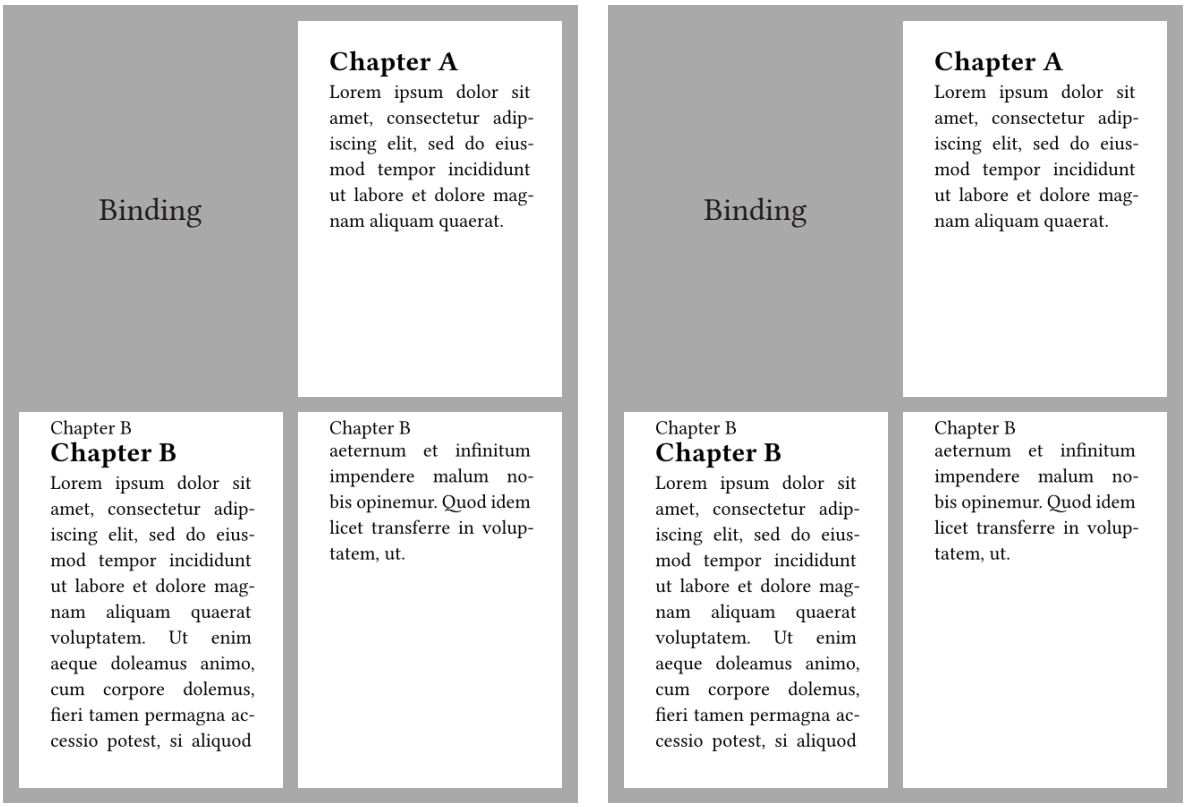

Figure 2: An example document showing `book: false` (left) and `book: true` (right).

This may produce unexpected results with Hydra is used outside the header and the text direction where it is used is different to where its anchor (see Section II.7) is placed. This can be prevented by explicitly setting its `binding` and `dir` arguments.

II.7. Anchoring

To use Hydra outside of the header, an anchor must be placed to get the correct active elements. Hydra will always use the last anchor it finds to search, it doesn't have to be inside the header, but should generally be, otherwise the behavior may be unexpected.

```
#import "@preview/hydra:0.5.0": hydra, anchor
#set page(header: anchor(), footer: context hydra())
```

Part III.
Reference

III.1. Stability

The following stability guarantees are made, this package tries to adhere to semantic versioning.

`unstable`	API may change with any version bump.
`stable`	API will not change without a major version bump or a minor version bump before `1.0.0`, if such a change occurs it is a bug and unintended.

III.2. Custom Types

The following custom types are used to pass around information easily, they are regular Typst dictionaries with expected keys.

III.2.1. queryable `stable`

Any type which can be used in `query`.

```
let queryable = label | function | selector
```

III.2.2. hydra-selector `stable`

Defines a selector for an ancestor or primary element.

```
let hydra-selector = (
  target: queryable,
  filter: ((hydra-context, candidates) => bool) | none,
)
```

III.2.3. full-selector `stable`

Defines a pair of primary and ancestor element selectors.

```
let full-selector = (
  primary: hydra-selector,
  ancestors: hydra-selector | none,
)
```

III.2.4. candidates `stable`

Defines the candidates that have been found in a specific context.

```
let candidates = (
  primary: (prev: content | none, next: content | none, last: content | none),
```

```
  ancestor: (prev: content | none, next: content | none),
)
```

III.2.5. hydra-context

unstable

Defines the options passed to Hydra and resolved contextual information needed for querying and displaying.

```
let hydra-context = (
  prev-filter: (hydra-context, candidates) => bool,
  next-filter: (hydra-context, candidates) => bool,
  display: (hydra-context, content) => content,
  skip-starting: bool,
  use-last: bool,
  book: bool,
  anchor: label | none,
  anchor-loc: location,
  primary: hydra-selector,
  ancestors: hydra-selector,
)
```

III.3. Modules

III.3.1. hydra

stable

The package entry point. All functions validate their inputs and panic using error messages directed at the end user.

```
#anchor()                    #hydra()
```

#anchor()

An anchor used to search from. When using `hydra` ouside of the page header, this should be placed inside the pge header to find the correct searching hydra-context. `hydra` always searches from the last anchor it finds, if and only if it detects that it is outside of the top-margin.

```
#hydra(
  ⟨prev-filter⟩: auto,
  ⟨next-filter⟩: auto,
  ⟨display⟩: auto,
  ⟨skip-starting⟩: true,
  ⟨use-last⟩: false,
  ⟨dir⟩: auto,
  ⟨binding⟩: auto,
  ⟨book⟩: false,
  ⟨anchor⟩: "<hydra-anchor>",
  ..⟨sel⟩
) ⟶ content
```

Query for an element within the bounds of its ancestors.

The hydra-context passed to various callbacks contains the resolved top-margin, the current location, as well as the binding direction, primary and ancestor element selectors and customized functions.

This function is contextual.

Argument

⟨prev-filter⟩: auto — function | auto

A function which receives the `hydra-context` and `candidates`, and returns if they are eligible for display. This function is called at most once. The primary next candidate may be none. If this is `auto` no filter is applied.

Argument

⟨next-filter⟩: auto — function | auto

A function which receives the `hydra-context` and `candidates`, and returns if they are eligible for display. This function is called at most once. The primary prev candidate may be none. If this is `auto` no filter is applied.

Argument

⟨display⟩: auto — function | auto

A function which receives the hydra-context and candidate element to display. If this is auto, the default implementaion will be used.

Argument

⟨skip-starting⟩: true — bool

Whether hydra should show the current candidate even if it's on top of the current page.

Argument

⟨use-last⟩: false — bool

If hydra should show the name of the first or last candidate on the page.

Argument

⟨dir⟩: auto — direction | auto

The reading direction of the document. If this is auto, the text direction is used. Be cautious about leaving this option on auto if you switch text direction mid-page and use hydra outside of footers or headers.

Argument

⟨binding⟩: auto — alignment | auto

The binding of the document. If this is auto, the binding is inferred from dir, similar to how it is done in page. Be cautious about leaving this on option on auto if you switch text direction mid-page and use hydra outside of footers or headers.

Argument

⟨book⟩: false — bool

The binding direction if it should be considered, none if not. If the binding direction is set it'll be used to check for redundancy when an element is visible on the last page. Make sure to set binding and dir if the document is not using left-to-right reading direction.

Argument

⟨anchor⟩: "<hydra-anchor>" — label | none

The label to use for the anchor if hydra is used outside the header. If this is none, the anchor is not searched.

Argument

..⟨sel⟩ — queryable | full-selector | int

The element to look for, to use other elements than headings, read the documentation on selectors. This can be an element function or selector, or an integer declaring a heading level.

III.3.2. core

unstable

The core logic module. Some functions may return results with error messages that can be used to panic or recover from instead of panicking themselves.

```
#display()                #get-text-dir()          #is-on-starting-page()
#execute()                #get-top-margin()        #locate-last-anchor()
#get-candidates()         #is-active-redundant()
#get-page-binding()       #is-active-visible()
```

#get-text-dir() ⟶ **direction**

Returns the current text direction.

This function is contextual.

#get-page-binding() ⟶ **alignment**

Returns the current page binding.

This function is contextual.

#get-top-margin() ⟶ **length**

Returns the current top margin.

This function is contextual.

#locate-last-anchor(⟨ctx⟩) ⟶ **location**

Get the last anchor location. Panics if the last anchor was not on the page of this `hydra-context`.

This function is contextual.

Argument

⟨ctx⟩ — `hydra-context`

The context from which to start.

#get-candidates(⟨ctx⟩, ⟨scope-prev⟩: true, ⟨scope-next⟩: true) ⟶ **candidates**

Get the element candidates for the given `hydra-context`.

This function is contextual.

Argument

⟨ctx⟩ — `hydra-context`

The context for which to get the candidates.

Argument

⟨scope-prev⟩: true — `bool`

Whether the search should be scoped by the first ancestor element in this direction.

Argument

⟨scope-next⟩: true — `bool`

Whether the search should be scoped by the first ancestor element in this direction.

`#is-on-starting-page(⟨ctx⟩, ⟨candidates⟩) ⟶ bool`

Checks if the current `hydra-context` is on a starting page, i.e. if the next candidates are on top of this `hydra-context`'s page.

This function is contextual.

Argument

`⟨ctx⟩` `hydra-context`

The context in which the visibility of the next candidates should be checked.

Argument

`⟨candidates⟩` `candidates`

The candidates for this `hydra-context`.

`#is-active-visible(⟨ctx⟩, ⟨candidates⟩) ⟶ bool`

Checks if the previous primary candidate is still visible.

This function is contextual.

Argument

`⟨ctx⟩` `hydra-context`

The hydra-context in which the visibility of the previous primary candidate should be checked.

Argument

`⟨candidates⟩` `candidates`

The candidates for this hydra-context.

`#is-active-redundant(⟨ctx⟩, ⟨candidates⟩) ⟶ bool`

Check if showing the active element would be redudnant in the current hydra-context.

This function is contextual.

Argument

`⟨ctx⟩` `hydra-context`

The hydra-context in which the redundancy of the previous primary candidate should be checked.

Argument

`⟨candidates⟩` `candidates`

The candidates for this hydra-context.

`#display(⟨ctx⟩)[⟨candidate⟩] ⟶ content`

Display a heading's numbering and body.

Argument

⟨ctx⟩ `hydra-context`

The hydra-context in which the element was found.

Argument

⟨candidate⟩ `content`

The heading to display, panics if this is not a heading.

`#execute(⟨ctx⟩)` ⟶ `content`

Execute the core logic to find and display elements for the given `hydra-context`. The `anchor-loc` of the context will be augmented.

This function is contextual.

Argument

⟨ctx⟩ `hydra-context`

The context for which to find and display the element.

III.3.3. selectors

stable

Contains functions used for creating custom selectors.

#by-level()	#custom()	#sanitize()

#custom(⟨element⟩, ⟨filter⟩: none, ⟨ancestors⟩: none, ⟨ancestors-filter⟩: none) ⟶ hydra-selector

Create a custom selector for `hydra`.

Argument
⟨element⟩ — queryable

The primary element to search for.

Argument
⟨filter⟩: none — function

The filter to apply to the element.

Argument
⟨ancestors⟩: none — queryable

The ancestor elements, this should match all of its ancestors.

Argument
⟨ancestors-filter⟩: none — function

The filter applied to the ancestors.

#by-level(⟨min⟩: none, ⟨max⟩: none, ..⟨exact⟩) ⟶ hydra-selector

Create a heading selector for a given range of levels.

Argument
⟨min⟩: none — int | none

The inclusive minimum level to consider as the primary heading.

Argument
⟨max⟩: none — int | none

The inclusive maximum level to consider as the primary heading.

Argument
..⟨exact⟩ — int | none

The exact level to consider as the primary element.

#sanitize(⟨name⟩, ⟨sel⟩, ⟨message⟩: auto) ⟶ hydra-selector

Turn a selector or function into a `hydra-selector`.

This function is considered unstable.

Argument

⟨name⟩ `str`

The name to use in the assertion message.

Argument

⟨sel⟩ `queryable` | `full-selector` | `int`

The selector to sanitize.

Argument

⟨message⟩: `auto` `str` | `auto`

The assertion message to use.

III.3.4. util

`unstable`

Utlity functions and values.

III.3.5. util/core

unstable

Utlity functions.

#or-default()	#page-binding()	#text-direction()

#or-default(⟨value⟩, ⟨default⟩, ⟨check⟩: none) ⟶ any

Substitute value for the return value of default() if it is a sentinel value.

Argument
⟨value⟩ — any

The value to check.

Argument
⟨default⟩ — function

The function to produce the default value with.

Argument
⟨check⟩: none — any

The sentinel value to check for.

#text-direction(⟨lang⟩) ⟶ direction

Returns the text direction for a given language, defaults to ltr for unknown languages.

Source: lang.rs#L50-L57

lang (str): The languge to get the text direction for.

#page-binding(⟨dir⟩) ⟶ alignment

Returns the page binding for a text direction.

Source: page.rs#L368-L373

dir (direction): The direction to get the page binding for.

#auto-or

An alias for or-default with check: auto.

#none-or

An alias for or-default with check: none.

#queryable-functions

A list of queryable element functions.

III.3.6. util/assert

unstable

Assertions used for input and state validation.

```
#element()                    #queryable()
#enum()                       #types()
```

#enum(⟨name⟩, ⟨value⟩, ..⟨expected-values⟩, ⟨message⟩: auto)

Assert that `value` is any of the given `expected-values`.

Argument

⟨name⟩ — str

The name use for the value in the assertion message.

Argument

⟨value⟩ — any

The value to check for.

Argument

..⟨expected-values⟩ — type

The expected variants of `value`.

Argument

⟨message⟩: auto — str | auto

The assertion message to use.

#types(⟨name⟩, ⟨value⟩, ..⟨expected-types⟩, ⟨message⟩: auto)

Assert that `value` is of any of the given `expected-types`.

Argument

⟨name⟩ — str

The name use for the value in the assertion message.

Argument

⟨value⟩ — any

The value to check for.

Argument

..⟨expected-types⟩ — type

The expected types of `value`.

Argument

⟨message⟩: auto — str | auto

The assertion message to use.

#element(⟨name⟩, ⟨element⟩, ..⟨expected-funcs⟩, ⟨message⟩: auto)

Assert that `element` is an element creatd by one of the given `expected-funcs`.

Argument

`⟨name⟩` `str`

The name use for the value in the assertion message.

Argument

`⟨element⟩` `any`

The value to check for.

Argument

`..⟨expected-funcs⟩` `type`

The expected element functions of `element`.

Argument

`⟨message⟩: auto` `str` | `auto`

The assertion message to use.

```
#queryable(⟨name⟩, ⟨value⟩, ⟨message⟩: auto)
```

Assert that `value` can be used in `query`.

Argument

`⟨name⟩` `str`

The name use for the value in the assertion message.

Argument

`⟨value⟩` `any`

The value to check for.

Argument

`⟨message⟩: auto` `str` | `auto`

The assertion message to use.

Part IV.
Index

A
#anchor 9

B
#by-level 14

C
#custom 14

D
#display 12

E
#element 18
#enum 18
#execute 13

G
#get-candidates 11
#get-page-binding 11
#get-text-dir 11
#get-top-margin 11

H
#hydra 9

I
#is-active-redundant 12
#is-active-visible 12
#is-on-starting-page 12

L
#locate-last-anchor 11

O
#or-default 17

P
#page-binding 17

Q
#queryable 19

S
#sanitize 14

T
#text-direction 17
#types 18